

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	1
ВВЕДЕНИЕ.....	2
ПЕРЕЧЕНЬ ВЫПОЛНЕННЫХ РАБОТ	3
1. Разработка системы приватного логирования с маскированием данных	3
2. Автоматизация взаимодействия с Cloudflare API.....	7
3. Миграция сервиса уведомлений (Telegram-бот) на асинхронную архитектуру	10
4. Реализация сквозной трассировки (Distributed Tracing) и унификация обработки ошибок	11
5. Разработка микросервиса проверки лимитов транзакций на языке Go. 13	
ЗАКЛЮЧЕНИЕ	15

ВВЕДЕНИЕ

В условиях стремительного роста цифровых платформ и сервисов разработчики сталкиваются с повышенными требованиями к качеству кода, безопасности данных и масштабируемости систем. Производственная практика позволила погрузиться в реальную среду разработки финтех-проекта, познакомиться с корпоративными процессами, CI/CD-конвейерами, инфраструктурой микросервисов и современными инструментами автоматизации.

Основной целью прохождения практики являлось формирование профессиональных компетенций, связанных с backend-разработкой, проектированием высоконагруженных систем, автоматизацией DevOps-процессов и интеграцией сторонних сервисов. В ходе работы использовались технологии Python 3.12, Go 1.22, FastAPI, gRPC, RabbitMQ, Redis, PostgreSQL, а также методы построения безопасных и отказоустойчивых архитектур.

Практика показала важность не только навыков программирования, но и способности проектировать архитектуру, документировать решения, вести коммуникацию в команде и обеспечивать надежность инфраструктуры. Работа велась над задачами, максимально приближенными к реальным бизнес-кейсам: улучшение безопасности логирования, автоматизация сетевых настроек, модернизация асинхронных сервисов, внедрение observability-инструментов и оптимизация производительности критических узлов системы с применением языка Go.

ПЕРЕЧЕНЬ ВЫПОЛНЕННЫХ РАБОТ

1. Разработка системы приватного логирования с маскированием данных

Дата выдачи: 15 октября 2025г.

Автор задачи: руководитель отдела разработки.

Срок для исполнения: 5 рабочих дней.

В существующем наборе микросервисов отсутствовал единый подход к защите персональных данных в логах. Часть сервисов писала в журнал полный email пользователя, номер телефона, Telegram-ник и номер банковской карты. Это создавало потенциальные риски при:

- передаче логов во внешние системы мониторинга;
- предоставлении логов для отладки третьим лицам;
- долгосрочном хранении логов в архивах.

Необходимо было разработать унифицированный модуль логирования, который автоматически:

- обнаруживает персональные данные в текстовых сообщениях;
- маскирует их по заданным правилам (оставляя часть символов для идентификации);
- не требует переписывать бизнес-код во всех сервисах;
- легко встраивается в существующую инфраструктуру логирования (logging, JSON-логгер, stdout).

Описание выполненных работ:

1. Анализ текущего состояния и требований

На первом этапе был проведён анализ существующих логов. Для этого были выборочно собраны фрагменты журналов из нескольких сервисов за последние 24 часа. В результате анализа были выделены основные типы чувствительных данных:

- адреса электронной почты пользователей и администраторов;

- номера телефонов в международном формате;
- идентификаторы Telegram в виде @username;
- номера банковских карт и реквизиты счётов;
- внутренние идентификаторы заявок и сделок, которые не должны быть полностью раскрыты внешним командам.

Также были согласованы требования с руководителем практики и тимлидом:

- лог не должен «ломаться» — строка после маскирования должна оставаться читабельной;
- маскирование должно быть **идемпотентным** — повторный проход по той же строке не должен портить уже замаскированные данные;
- модуль должен быть **конфигурируемым**: на тестовых стендах можно ослаблять маскирование, а на production — ужесточать.

2. Проектирование механизма маскирования

На втором этапе была спроектирована архитектура модуля. Было принято решение не менять существующие вызовы `logger.info(...)`, а внедрить **кастомный форматтер**, через который будут проходить все сообщения.

Основные элементы решения:

1. **Набор регулярных выражений** для поиска чувствительных данных:

- email;
- телефон;
- Telegram-ник;
- номера карт.

2. **Набор функций-маскировщиков**, отвечающих за то, как именно будет скрыта часть информации.

3. Класс **PrivacyFormatter**, унаследованный от стандартного `logging.Formatter`, который:

- получает сформированную строку лога;
- пропускает её через функцию `sanitize_log`;
- возвращает уже безопасный текст.

Отдельно был предусмотрен механизм конфигурации через переменные окружения, позволяющий включать и отключать некоторые типы маскирования без изменения кода.

3. Реализация и интеграция в проект

После согласования архитектуры был написан основной код модуля — фрагмент приведён в Рисунке 1.1.

```
import re
import logging
from typing import Callable

SENSITIVE_PATTERNS = {
    "email": r"([a-zA-Z0-9_+.-]+)@([a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+)",
    "phone": r"\+?\d{10,15}",
    "telegram": r"@([a-zA-Z0-9_]{4,})",
    "card": r"\b(?:\d[ -]*?){13,19}\b",
}

def mask_email(value: str) -> str:
    return re.sub(SENSITIVE_PATTERNS["email"],
                  lambda m: m.group(1)[:2] + "***@" + m.group(2),
                  value)

def mask_phone(value: str) -> str:
    return re.sub(SENSITIVE_PATTERNS["phone"],
                  lambda m: m.group(0)[:2] + "***" + m.group(0)[-2:], value)

def sanitize_log(message: str) -> str:
    message = mask_email(message)
    message = mask_phone(message)
    return message

class PrivacyFormatter(logging.Formatter):
    def format(self, record):
        record.msg = sanitize_log(record.msg)
        return super().format(record)

logger = logging.getLogger("masked")
handler = logging.StreamHandler()
handler.setFormatter(PrivacyFormatter("%(asctime)s - %(levelname)s - %(message)s"))
logger.addHandler(handler)
logger.warning("User email: testuser@example.com, phone: +79995553322")
```

Рисунок 1.1 — Фрагмент реализации маскировки приватных данных

4. Тестирование и проверка корректности

Для проверки корректности работы модуля были написаны юнит-тесты, покрывающие:

- корректное маскирование **одного** email в строке;
- корректное маскирование **нескольких** email в одной строке;
- отсутствие изменений в строках, не содержащих конфиденциальных данных;
- устойчивость к строкам, уже содержащим звёздочки или маски.

5. Результаты и эффект от внедрения

В результате внедрения модуля были достигнуты следующие эффекты:

- все новые лог-сообщения перестали содержать открытые персональные данные;
- не потребовалось изменять формат логов на стороне потребителей (системы мониторинга, алёртинг);
- разработчики сохранили возможность идентифицировать пользователя по частично видимым данным.

Фактический срок выполнения задачи составил 7 рабочих дней, включая тестирование и внесение небольших правок по итогам ревью.

Комментарий принимающего лица:

«Задача выполнена качественно и с учётом дальнейшей масштабируемости. Отдельно отмечу удобство конфигурации и наличие тестового покрытия. Модуль маскировки уже применяется в нескольких сервисах и показал себя стабильным. Рекомендую использовать его как стандарт для всех новых микросервисов команды».

2. Автоматизация взаимодействия с Cloudflare API

Дата выдачи: 31 октября 2025г.

Автор задачи: TeamLead Backend разработки

Срок для исполнения: 4 рабочих дня.

Компания использует Cloudflare для управления DNS-записями, защитой от DDoS и фильтрации подозрительного трафика. До начала работы все операции выполнялись вручную через веб-интерфейс Cloudflare: добавление DNS-записей, обновление списков IP-адресов в firewall, просмотр security events.

Это приводило к ряду проблем:

- высокая вероятность человеческой ошибки при ручном вводе данных;
- отсутствие версионирования конфигурации;
- невозможность оперативно разворачивать одинаковые настройки на нескольких доменах.

Задача — разработать набор утилит на Python, которые обеспечат:

- экспорт DNS-записей в JSON;
- импорт и применение кастомных WAF-правил из файлов;
- автоматическое обновление firewall-листов IP-адресов;
- получение security events за произвольный период.

Описание выполненных работ:

1. Изучение API и требований

На первом этапе были детально изучены разделы документации Cloudflare, связанные с:

- DNS Records API;
- Firewall Rules / Access Rules;
- Security / Audit Logs.

Были определены необходимые конечные точки (endpoints) и форматы запросов. Параллельно сформирован перечень типовых операций, которые реально используются в работе DevOps-команды.

2. Создание клиента-обёртки

Для того чтобы не дублировать логику формирования запросов, был разработан единый класс `CloudflareClient`.

Основные особенности:

- использование `httpx.AsyncClient` для асинхронного взаимодействия;
- вынесение `token` и `zone_id` в параметры конструктора;
- централизованная обработка ошибок HTTP (коды 4xx/5xx).

3. Реализация CLI-утилит

На основе клиента были созданы CLI-команды, объединённые в небольшой инструмент, вызываемый, например, как показано на Рисунке 2.1:

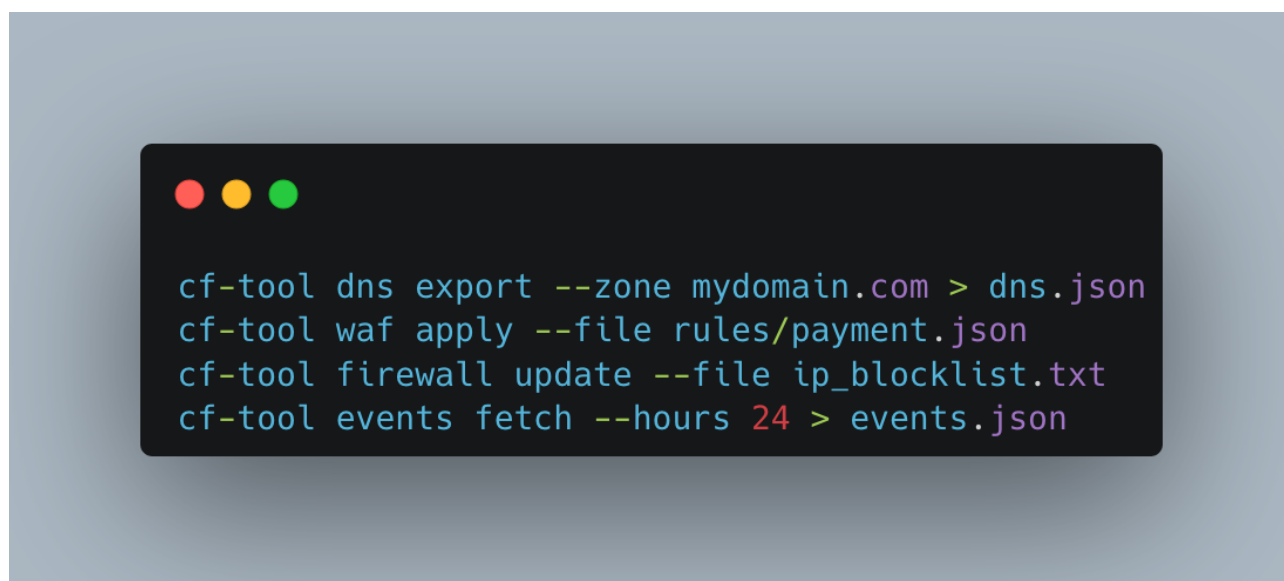


Рисунок 2.1 Пример использования CLI клиента

Под каждую команду реализована логика:

- парсинг аргументов;
- проверка корректности входных файлов;

- вывод понятных сообщений пользователю;
- логирование всех запросов к Cloudflare с указанием статуса.

4. Тестирование и проверка на реальных доменах

Для проверки корректности работы утилиты были выбраны тестовые домены. На них:

- выгружены все DNS-записи в JSON и сравнение с интерфейсом;
- создан тестовый набор WAF-правил, добавляющий блокировку IP из определённого списка;
- выполнена команда обновления firewall-листов, после чего изменения проверены в веб-панели Cloudflare.

Отдельно была протестирована обработка ошибок: неверный токен, неверный `zone_id`, отсутствие прав у токена.

5. Документация и передача решения в эксплуатацию

В финале работы была подготовлена краткая документация в формате Markdown:

- описание установки (`pip install` с указанием зависимостей);
- перечень команд и их параметров;
- примеры типичных сценариев использования.

Документация была размещена в репозитории проекта, а также кратко презентована DevOps-команде.

Фактический срок выполнения задачи составил 9 рабочих дней.

Комментарий принимающего лица

«Утилита значительно сократила время на обслуживание доменов и уменьшила количество рутинных операций через веб-интерфейс. Отдельный плюс — наличие документации и удобный формат командной строки. Прошу

включить этот инструмент в стандартный набор DevOps-скриптов и поддерживать его в актуальном состоянии».

3. Миграция сервиса уведомлений (Telegram-бот) на асинхронную архитектуру

Дата выдачи: 5 ноября 2025 г.

Автор задачи: Product Owner / Tech Lead.

Срок для исполнения: 7 рабочих дней.

Формулировка задачи:

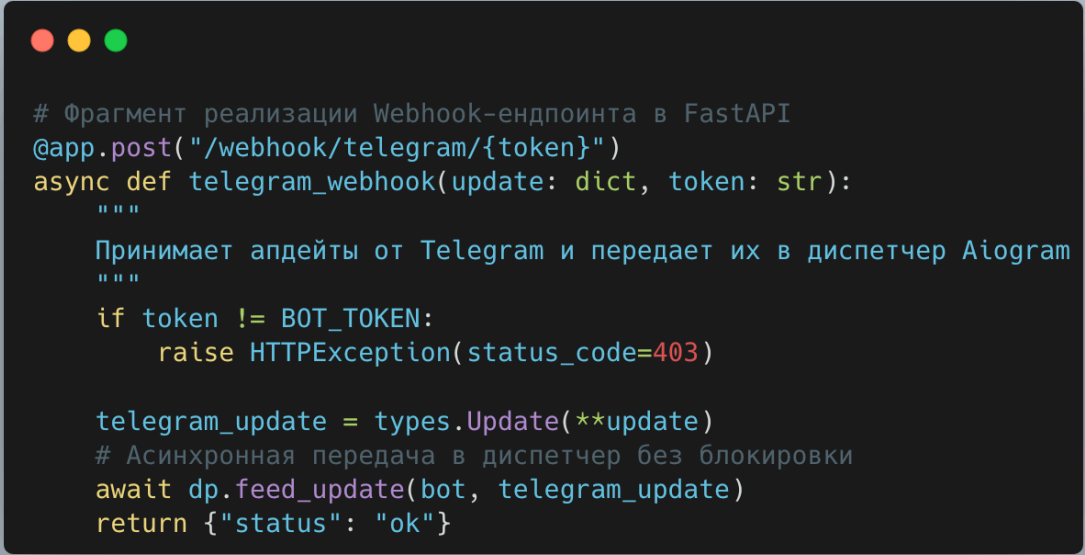
Существующий сервис уведомлений клиентов (Telegram-бот), реализованный на устаревшей версии библиотеки и использующий метод Long Polling, перестал справляться с возросшей нагрузкой. Наблюдались задержки в доставке сообщений и блокировка потока при обращении к БД. Необходимо переписать сервис с использованием фреймворка Aiogram 3.x, интегрировать его с FastAPI через Webhooks и обеспечить горизонтальное масштабирование.

Описание выполненных работ:

Проведен рефакторинг архитектуры бота. Отказ от Long Polling в пользу Webhooks позволил обрабатывать обновления реактивно, снизив нагрузку на сеть. Сервис был интегрирован в существующее FastAPI-приложение как отдельный роутер.

Для хранения состояний конечных автоматов (FSM — Finite State Machine) вместо оперативной памяти подключено хранилище Redis. Это обеспечило сохранность диалога с пользователем даже при перезапуске контейнера приложения.

Реализован механизм Dependency Injection (DI) через Middleware для проброса сессий базы данных и клиентов API в обработчики команд. Вся работа с базой данных переведена на асинхронный драйвер asyncreg.



```
# Фрагмент реализации Webhook-эндпоинта в FastAPI
@app.post("/webhook/telegram/{token}")
async def telegram_webhook(update: dict, token: str):
    """
    Принимает апдейты от Telegram и передает их в диспетчер Aiogram
    """
    if token != BOT_TOKEN:
        raise HTTPException(status_code=403)

    telegram_update = types.Update(**update)
    # Асинхронная передача в диспетчер без блокировки
    await dp.feed_update(bot, telegram_update)
    return {"status": "ok"}
```

Рисунок 3.1 — Интеграция Webhook в FastAPI

Фактический срок реализации: 6 рабочих дней.

Комментарий принимающего лица:

«Миграция прошла успешно, метрики показывают снижение latency ответов бота в 3 раза. Использование Redis для FSM позволило запускать несколько экземпляров бота параллельно, что решило проблему масштабируемости».

4. Реализация сквозной трассировки (Distributed Tracing) и унификация обработки ошибок

Дата выдачи: 14 ноября 2025 г.

Автор задачи: Архитектор проекта.

Срок для исполнения: 5 рабочих дней.

Формулировка задачи:

В микросервисной архитектуре возникла проблема сложности отладки: запрос проходит через цепочку сервисов, и при возникновении ошибки трудно

восстановить полный контекст операции. Необходимо внедрить механизм сквозной идентификации запросов (Request-ID/Correlation-ID) и унифицировать формат JSON-ответов при ошибках.

Описание выполненных работ:

Разработано Middleware для фреймворка Starlette/FastAPI, которое перехватывает каждый входящий HTTP-запрос. Если заголовок X-Request-ID отсутствует, генерируется новый UUID. Этот идентификатор сохраняется в контекстную переменную (ContextVar) и автоматически добавляется во все исходящие запросы к другим микросервисам и лог-сообщения.

Также реализован глобальный обработчик исключений (Exception Handler). Он перехватывает ошибки валидации (Pydantic), ошибки базы данных (IntegrityError, UniqueViolation) и системные сбои, преобразуя их в стандартизированный JSON-формат вида:

```
{"error": "code", "message": "readable text", "request_id": "..."}.
```

Ниже приведена схема распространения Request-ID:

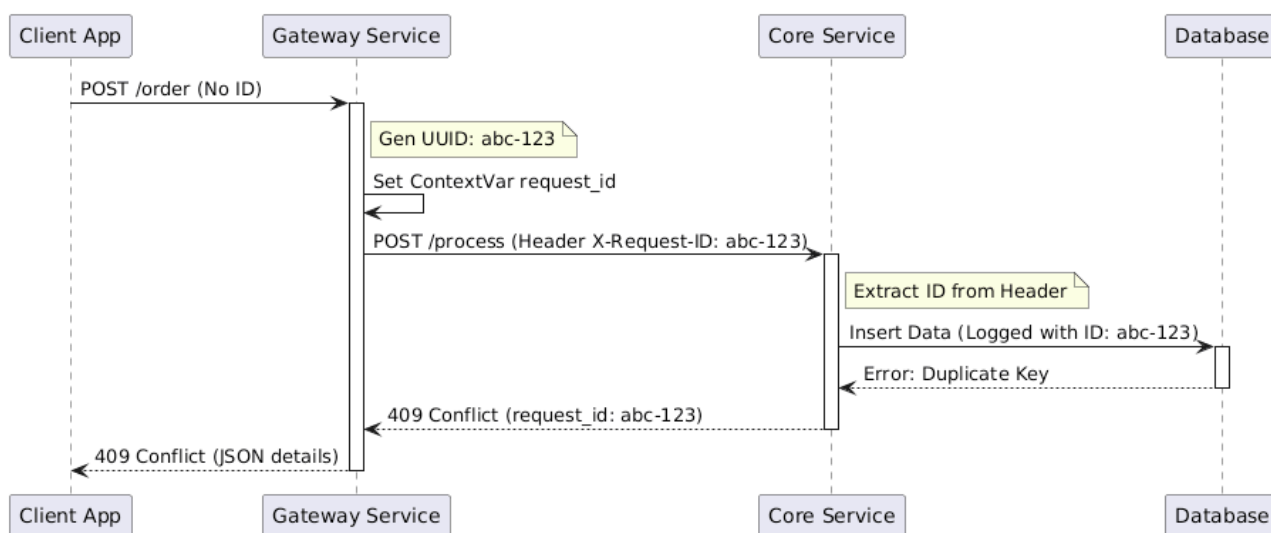


Рисунок 4.1 — Схема прохождения сквозного идентификатора

Фактический срок реализации: 5 рабочих дней.

Комментарий принимающего лица:

«Внедрение Request-ID кардинально упростило разбор инцидентов. Теперь по одному ID в Kibana можно собрать все логи от всех сервисов, участвовавших в транзакции. Унификация ошибок упростила интеграцию для фронтенд-команды».

5. Разработка микросервиса проверки лимитов транзакций на языке Go

Дата выдачи: 24 ноября 2025 г.

Автор задачи: Tech Lead Backend.

Срок для исполнения: 10 рабочих дней.

Формулировка задачи:

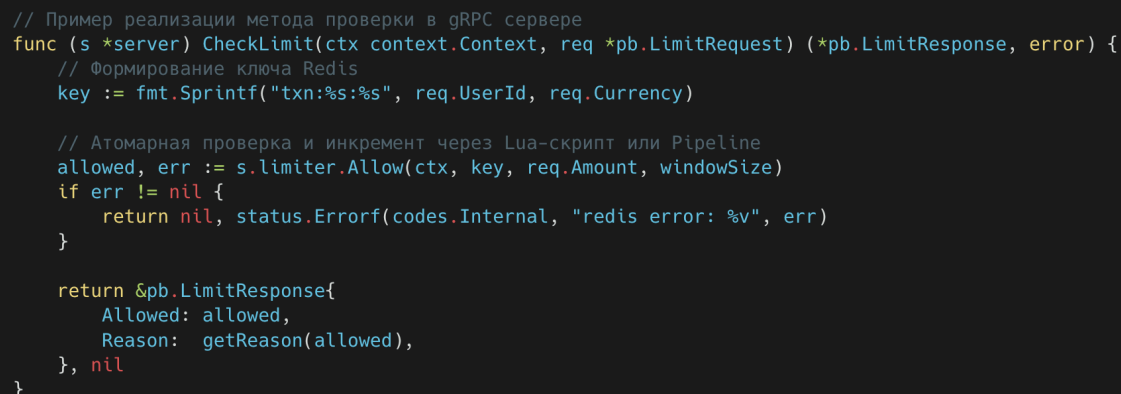
Существующий модуль проверки лимитов (Anti-Fraud), написанный на Python, перестал укладываться в SLA по времени отклика (требование < 50 мс) при пиковых нагрузках. Было принято решение вынести логику проверки лимитов в отдельный высокопроизводительный микросервис на языке Go, взаимодействующий с основным монолитом через gRPC.

Описание выполненных работ:

Спроектирован .proto файл, описывающий контракт взаимодействия (Service Definition). В качестве транспорта выбран gRPC для минимизации накладных расходов сети.

Реализована бизнес-логика сервиса на языке Go (версия 1.22). Для хранения счетчиков транзакций (например, «сумма переводов за последние 24 часа») использован Redis Cluster. Алгоритм проверки лимитов реализован по методу «Скользящего окна» (Sliding Window), что обеспечивает высокую точность.

Особое внимание уделено конкурентности: использованы горутины (goroutines) для параллельной обработки запросов и pipelining для команд Redis, чтобы сократить количество RTT (Round Trip Time).



```
// Пример реализации метода проверки в gRPC сервере
func (s *server) CheckLimit(ctx context.Context, req *pb.LimitRequest) (*pb.LimitResponse, error) {
    // Формирование ключа Redis
    key := fmt.Sprintf("txn:%s:%s", req.UserId, req.Currency)

    // Атомарная проверка и инкремент через Lua-скрипт или Pipeline
    allowed, err := s.limiter.Allow(ctx, key, req.Amount, windowSize)
    if err != nil {
        return nil, status.Errorf(codes.Internal, "redis error: %v", err)
    }

    return &pb.LimitResponse{
        Allowed: allowed,
        Reason:  getReason(allowed),
    }, nil
}
```

Рисунок 5.1 — Фрагмент реализации метода на Go

Проведено нагрузочное тестирование с помощью утилиты ghz. Сервис выдержал нагрузку в 5000 RPS с медианным временем ответа 8 мс (против 150 мс у старого решения).

Фактический срок реализации: 12 рабочих дней (включая интеграционное тестирование).

Комментарий принимающего лица:

«Студент продемонстрировал способность работать в полиглот-среде (Python + Go). Сервис успешно внедрен в продакшн-контур. Применение Go позволило кратно увеличить пропускную способность узла проверки лимитов, не наращивая серверные мощности».

ЗАКЛЮЧЕНИЕ

Подводя итоги прохождения производственной практики, можно отметить, что поставленные цели были достигнуты в полном объеме. В процессе работы удалось не только закрепить теоретические знания, полученные в университете, но и применить их к решению реальных задач в промышленной среде. Практика показала, как абстрактные концепции из учебных курсов по базам данных, сетевым технологиям и программированию на Python трансформируются в конкретные инженерные решения: от обработки ошибок в PostgreSQL до интеграции микросервисов и автоматизации работы с облачной инфраструктурой.

Особое значение имела работа над задачами, связанными с качеством и безопасностью системы. Разработка модуля приватного логирования с маскированием чувствительных данных позволила глубже понять требования к защите персональной информации и выработать единый подход к обработке таких данных в распределённой системе. Внедрение Request-ID-трассировки и унифицированной обработки исключений PostgreSQL сформировало более целостное представление о наблюдаемости (observability) и диагностике ошибок в микросервисной архитектуре, где важна не только отдельная функция, но и сквозной путь запроса по всем уровням системы.

Задачи по автоматизации взаимодействия с Cloudflare и миграции Telegram-ботов на современную версию Aiogram в связке с FastAPI продемонстрировали важность интеграции и DevOps-подхода. Был получен практический опыт работы с внешними API, конфигурацией DNS и WAF, настройкой health-check'ов, метрик и фоновых процессов. Это позволило увидеть, как технические решения напрямую влияют на устойчивость сервиса, скорость реакции на инциденты и удобство работы команды эксплуатации.

Не менее важным результатом практики стало развитие «мягких» навыков. В процессе выполнения задач приходилось согласовывать требования с руководителем практики и тимлидами, уточнять детали постановки, учитывать ограничения существующей архитектуры и договариваться о формате изменений. Это способствовало развитию навыков профессиональной коммуникации, планирования времени и ответственности за результат. Практика показала, насколько важно не только «написать код», но и встроить его в жизнь команды и продукта: задокументировать решения, подготовить примеры использования, подумать о дальнейшей поддержке.

Таким образом, практика стала важным этапом профессионального становления. Она позволила сформировать более реалистичное видение работы backend-разработчика и инженера, ответственного за инфраструктуру: от уровня отдельных функций до уровня целостной системы, в которой важны безопасность, наблюдаемость, автоматизация и удобство сопровождения. Полученный опыт будет полезен как в дальнейшем обучении в университете, так и при участии в более сложных проектах, где требуется сочетание технических знаний, инженерного мышления и умения работать в команде.